Race Timer
BLE Integration

iOS App ↔ Mainboard Communication Protocol

System Architecture

- iOS App sends commands → Mainboard
- Mainboard sends events (notifications) → iOS App
- iOS App can read the current race status at any time
- Mainboard handles all timing with hardware TIMER
- 4 race buttons + 1 auxiliary button (start/stop)
- 4-digit seven-segment display (MAX7221 via SPI)
- 5 button LEDs + 1 status LED

Key Design Principles

1. Mainboard is the source of truth for elapsed time.
2. All times are in milliseconds (uint32_t, little-endian).
3. The race continues even if BLE disconnects mid-race (error state).
4. Button 5 allows standalone operation without the iOS app.
5. The iOS app should subscribe to notifications before starting a race.

1. BLE Connection Details

Parameter	Value
Device Name	RaceTimer
Appearance	768 (0x0300)
Advertising Type	Connectable, Undirected
Max Connections	5
Max Paired Devices	5
BLE Features	SMP (Secure Simple Pairing)
Advertising Data	Flags + 128-bit Service UUID
Scan Response	Complete Local Name ("RaceTimer")

2. GATT Service & Characteristics

Service UUID

`00001000-7261-6365-7469-6D6572303031`

Characteristics

Characteristic	UUID	Properties	Direction
Command	00001001-7261-6365-7469-6D6572303031	Write / Write Without Response	iOS → Mainboard
Event	00001002-7261-6365-7469-6D6572303031	Notify	Mainboard → iOS
Status	00001003-7261-6365-7469-6D6572303031	Read	Mainboard → iOS

Characteristic Details

Command Characteristic (Write)

The iOS app writes to this characteristic to send commands to the mainboard. Supports both Write and Write Without Response. Each write must contain a 4-byte command packet.

Event Characteristic (Notify)

The mainboard sends real-time race events via BLE notifications on this characteristic. The iOS app MUST enable notifications (subscribe) on this characteristic before sending any race commands. Each notification contains an 11-byte event packet.

⚠ **IMPORTANT:** Enable notifications on the Event characteristic immediately after connecting, before sending any commands.

Status Characteristic (Read)

A readable characteristic that returns the current race state as a single byte (uint8). The iOS app can read this at any time to check the mainboard's current state.

3. Command Protocol (iOS → Mainboard)

Command Packet Structure (4 bytes)

Byte	Field	Type	Description
0	cmd_type	uint8	Command ID (see table below)
1	mode	uint8	Race mode (for CMD_SET_MODE only, otherwise 0x00)
2	reserved	uint8	Reserved — set to 0x00
3	reserved	uint8	Reserved — set to 0x00

Command IDs

Command	ID (Hex)	Description	Valid States	Response Event
CMD_START_RACE	0x01	Start the race timer	CONFIGURED, ARMED	EVT_START_ACK
CMD_NEW_RACE	0x02	Reset and prepare for a new race	Any	(state changes to IDLE)
CMD_RESET	0x03	Full reset (same as NEW_RACE)	Any	(state changes to IDLE)
CMD_PING	0x04	Ping — check mainboard is alive	Any	EVT_PONG
CMD_GET_STATUS	0x05	Request current race status	Any	EVT_STATUS
CMD_SET_MODE	0x06	Set the race mode (byte 1 = mode)	IDLE, CONFIGURED	(state → CONFIGURED)

Command Examples (Raw Bytes)

Set Mode to 4-Contestant (Mode 1):

```
[0x06, 0x01, 0x00, 0x00]   // CMD_SET_MODE, mode=0x01
```

OR

Set Mode to 1-Contestant / 4-Segment (Mode 2):

```
[0x06, 0x02, 0x00, 0x00]   // CMD_SET_MODE, mode=0x02
```

Start the Race:

```
[0x01, 0x00, 0x00, 0x00]   // CMD_START_RACE
```

Ping:

```
[0x04, 0x00, 0x00, 0x00]   // CMD_PING
```

Get Status:

```
[0x05, 0x00, 0x00, 0x00]   // CMD_GET_STATUS
```

New Race / Reset:

```
[0x02, 0x00, 0x00, 0x00]   // CMD_NEW_RACE
```

4. Event Protocol (Mainboard → iOS)

Event Packet Structure (11 bytes)

Byte(s)	Field	Type	Description
0	event_type	uint8	Event ID (see table below)
1	mode	uint8	Current race mode (0x01 or 0x02)
2	button_index	uint8	Button number (1–4), or 0 if N/A
3–6	elapsed_time_ms	uint32 (LE)	Elapsed time in milliseconds (little-endian)
7–10	race_id	uint32 (LE)	Unique race identifier (little-endian)

Note: All multi-byte integers are in little-endian byte order (standard for BLE / ARM).

Event IDs

Event	ID (Hex)	Description	Key Fields
EVT_START_ACK	0x10	Race has started — timer is now running	elapsed_time_ms = 0, race_id = new ID
EVT_STOP_EVENT	0x11	A button was pressed during the race	button_index = 1–4, elapsed_time_ms = time of press
EVT_RACE_COMPLETE	0x12	Race is fully complete	button_index = winner (Mode 1) or 0, elapsed_time_ms = final time
EVT_STATUS	0x13	Current status response (to CMD_GET_STATUS)	elapsed_time_ms = current elapsed time
EVT_PONG	0x14	Ping response (mainboard is alive)	elapsed_time_ms = 0
EVT_ERROR	0x1F	Error occurred (invalid command, etc.)	all fields may be 0

Parsing Example (11-byte notification)

When you receive a notification on the Event characteristic, parse the 11 bytes:

```
App Sends [0x06, 0x01, 0x00, 0x00]   // CMD_SET_MODE, mode=0x01

Race Starts App Sends [0x01, 0x00, 0x00, 0x00]   // CMD_START_RACE


When first time button pressed after race start
Example received bytes: [0x11, 0x01, 0x02, 0xE8, 0x03, 0x00, 0x00, 0xAB, 0xCD, 0x00, 0x00]

Parsed:
  event_type      = 0x11 → EVT_STOP_EVENT
  mode             = 0x01 → Mode 1 (4 contestants)
  button_index     = 0x02 → Button 2
  elapsed_time_ms  = 0x000003E8 → 1000 ms (1.000 seconds)
  race_id          = 0x0000CDAB → 52651

When second time button pressed after race start

Example received bytes: [0x11, 0x01, 0x01, 0x88, 0x13, 0x00, 0x00, 0xAB, 0xCD, 0x00, 0x00]

Parsed:
  event_type      = 0x11 → EVT_STOP_EVENT
  mode             = 0x01 → Mode 1 (4 contestants)
  button_index     = 0x02 → Button 1
  elapsed_time_ms  = 0x00001388 → 5000 ms (5.000 seconds)
  race_id          = 0x0000CDAB → 52651
```

Race completed with winner

```
Example received bytes: [0x12, 0x01, 0x02, 0xE8, 0x03, 0x00, 0x00, 0xAB, 0xCD, 0x00, 0x00]

Parsed:
  event_type      = 0x11 → EVT_RACE_COMPLETE
  mode             = 0x01 → Mode 1 (4 contestants)
  button_index     = 0x02 → Button 2
  elapsed_time_ms  = 0x000003E8 → 1000 ms (1.000 seconds)
  race_id          = 0x0000CDAB → 52651


[0x02, 0x00, 0x00, 0x00]   // CMD_NEW_RACE

App Sends [0x06, 0x01, 0x00, 0x00]   // CMD_SET_MODE, mode=0x01

Race Starts App Sends [0x01, 0x00, 0x00, 0x00]   // CMD_START_RACE
```

5. Race States

State Machine

The mainboard follows a strict state machine. The Status characteristic reflects the current state value.

State	Value	Description	LED Indicators
DISCONNECTED	0	No BLE connection	Status LED: fast blink (200ms on / 800ms off), Button LEDs: OFF
IDLE	1	BLE connected, no race configured	Status LED: solid ON, Button LEDs: OFF, Display: "----"
CONFIGURED	2	Mode set, ready to start	Status LED: solid ON, Display: "00:00"
ARMED	3	Ready to start race	Status LED: solid ON, Button LEDs: ON for active lanes
RUNNING	4	Race in progress, timer counting	Status LED: solid ON, Display: running elapsed time
FINISHED	5	Race complete, results available	Status LED: solid ON, Winner button LED: ON
ERROR	6	Error (e.g. BLE lost during race)	Status LED: rapid blink (100ms/100ms), Display: "----"

State Transitions

DISCONNECTED → IDLE (BLE connects)
IDLE → CONFIGURED (CMD_SET_MODE received)
CONFIGURED → RUNNING (CMD_START_RACE received)
ARMED → RUNNING (CMD_START_RACE received)
RUNNING → FINISHED (all buttons pressed or manual stop)
RUNNING → ERROR (BLE disconnects during race)
ERROR → RUNNING (BLE reconnects, timer still running)
ERROR → IDLE (BLE reconnects, timer stopped)
FINISHED → IDLE (CMD_NEW_RACE / CMD_RESET)
Any State → IDLE (CMD_RESET)
Any State → DISCONNECTED (BLE disconnects, except during race)

6. Race Modes

Mode 1: 4-Contestant Race (0x01)

- 4 independent contestants, each assigned one button (1–4).
- All buttons are active. The first button pressed is the winner.
- After the winner is declared, the remaining buttons can still be pressed to record 2nd, 3rd, and 4th place times.
- A 30-second grace period runs after the first button press:
  - If all 4 buttons are pressed within 30 seconds, race ends immediately.
  - If the grace period expires, race auto-finishes with available results.
- Button 5 (auxiliary) can manually stop the race at any time.
- Each EVT_STOP_EVENT includes the button_index and elapsed_time_ms.
- The final EVT_RACE_COMPLETE includes the winner's button and time.

Mode 2: 1-Contestant / 4-Segment Race (0x02)

- 1 contestant completes 4 sequential segments.
- Buttons 1–4 are pressed in order as the contestant passes each checkpoint.
- Each button press records a segment/split time.
- The race completes when all 4 buttons have been pressed.
- EVT_STOP_EVENT is sent for each segment with elapsed_time_ms from race start.
- EVT_RACE_COMPLETE is sent when all 4 segments are done, with total elapsed time.
- Button 5 can manually stop/end the race at any time.

Mode	Value	Contestants	Buttons Used	Completion Condition
4-Contestant	0x01	4	Buttons 1–4 (independent)	All 4 pressed, or 30s grace after 1st
1-Contestant	0x02	1	Buttons 1–4 (sequential)	All 4 segments completed

7. Typical Flow Examples

Example 1: Mode 1 — 4-Contestant Race

Step-by-step sequence for a typical 4-contestant race:

Step	Action	Direction	Bytes / Event
1	iOS scans & connects to "RaceTimer"	—	State → IDLE
2	iOS enables notifications on Event char	iOS → FW	Write to CCCD: [0x01, 0x00]
3	iOS sets Mode 1	iOS → FW	Write: [0x06, 0x01, 0x00, 0x00]
4	(State changes to CONFIGURED)	—	Status char = 0x02
5	iOS sends Start Race	iOS → FW	Write: [0x01, 0x00, 0x00, 0x00]
6	FW acknowledges start	FW → iOS	Notify: EVT_START_ACK (0x10)
7	Contestant 3 presses button first	FW → iOS	Notify: EVT_STOP_EVENT, btn=3, time=2345ms
8	Contestant 1 presses button	FW → iOS	Notify: EVT_STOP_EVENT, btn=1, time=2890ms
9	Contestant 4 presses button	FW → iOS	Notify: EVT_STOP_EVENT, btn=4, time=3102ms
10	Contestant 2 presses button	FW → iOS	Notify: EVT_STOP_EVENT, btn=2, time=3500ms
11	Race complete (all 4 buttons)	FW → iOS	Notify: EVT_RACE_COMPLETE, winner=3, time=2345ms
12	iOS sends New Race to reset	iOS → FW	Write: [0x02, 0x00, 0x00, 0x00]

Example 2: Mode 2 — 1-Contestant / 4 Segments

Step	Action	Direction	Bytes / Event
1	iOS connects and enables notifications	—	(same as above)
2	iOS sets Mode 2	iOS → FW	Write: [0x06, 0x02, 0x00, 0x00]
3	iOS sends Start Race	iOS → FW	Write: [0x01, 0x00, 0x00, 0x00]
4	FW acknowledges start	FW → iOS	Notify: EVT_START_ACK (0x10)
5	Segment 1 checkpoint (Button 1)	FW → iOS	Notify: EVT_STOP_EVENT, btn=1, time=15230ms
6	Segment 2 checkpoint (Button 2)	FW → iOS	Notify: EVT_STOP_EVENT, btn=2, time=31540ms
7	Segment 3 checkpoint (Button 3)	FW → iOS	Notify: EVT_STOP_EVENT, btn=3, time=48700ms
8	Segment 4 — finish (Button 4)	FW → iOS	Notify: EVT_STOP_EVENT, btn=4, time=62100ms
9	Race complete (all 4 segments)	FW → iOS	Notify: EVT_RACE_COMPLETE, btn=0, time=62100ms

8. Error Handling

BLE Disconnection During Race

• If BLE disconnects while a race is RUNNING, the mainboard enters ERROR state but the timer keeps running.
• Buttons still work — results are still recorded locally on the mainboard.
• If the iOS app reconnects, the mainboard sends EVT_STATUS with the current elapsed time.
• The state transitions back to RUNNING if the timer is still active.
• Button 5 can still be used to manually stop the race even without BLE.

Invalid Commands

• Sending CMD_START_RACE in a state other than CONFIGURED or ARMED → EVT_ERROR.
• Sending CMD_SET_MODE in a state other than IDLE or CONFIGURED → ignored (warning logged).
• Sending an unknown command ID → EVT_ERROR.
• Command packet shorter than 4 bytes → BLE ATT error (invalid attribute length).

Duplicate Button Presses

• If a button that has already been recorded is pressed again during a race, the duplicate press is ignored.
• Only the first press per button is recorded and reported via EVT_STOP_EVENT.

Reconnection Best Practices

1. After reconnecting, re-enable notifications on the Event characteristic.
2. Send CMD_GET_STATUS (0x05) to check the current state and elapsed time.
3. Read the Status characteristic to confirm the mainboard's state.
4. If the race was running, sync the UI with the received elapsed time.
5. If the race finished while disconnected, the FINISHED state will be returned.

9. Status Characteristic

The Status characteristic is a readable byte that maps directly to the race state:

Value	State	Meaning
0x00	DISCONNECTED	BLE not connected (you won't read this since you're connected)
0x01	IDLE	Connected, waiting for mode/configuration
0x02	CONFIGURED	Mode set, ready to start
0x03	ARMED	Armed and ready
0x04	RUNNING	Race in progress
0x05	FINISHED	Race done, results available
0x06	ERROR	Error state (e.g., previous BLE disconnect)

10. Quick Reference Card

UUIDs

```
Service:    00001000-7261-6365-7469-6D6572303031
Command:    00001001-7261-6365-7469-6D6572303031  (Write)
Event:      00001002-7261-6365-7469-6D6572303031  (Notify)
Status:     00001003-7261-6365-7469-6D6572303031  (Read)
```

Commands (4 bytes: [cmd, mode, 0x00, 0x00])

```
START_RACE  = 0x01    →  Response: EVT_START_ACK (0x10)
NEW_RACE    = 0x02    →  Resets to IDLE
RESET       = 0x03    →  Resets to IDLE
PING        = 0x04    →  Response: EVT_PONG (0x14)
GET_STATUS  = 0x05    →  Response: EVT_STATUS (0x13)
SET_MODE    = 0x06    →  byte[1] = 0x01 (4-player) or 0x02 (1-player)
```

Events (11 bytes: [evt, mode, btn, time(4), race_id(4)])

```
START_ACK     = 0x10    race started
STOP_EVENT    = 0x11    button pressed (btn=1-4, time=elapsed ms)
RACE_COMPLETE = 0x12    race finished (btn=winner, time=final ms)
STATUS        = 0x13    status response (time=current elapsed ms)
PONG          = 0x14    alive response
ERROR         = 0x1F    error occurred
```

Minimal Race Flow

```
1. Connect to "RaceTimer"
2. Discover service 00001000-...
3. Enable notifications on Event char (00001002-...)
4. Write CMD_SET_MODE:   [0x06, 0x01, 0x00, 0x00]
5. Write CMD_START_RACE: [0x01, 0x00, 0x00, 0x00]
6. Listen for EVT_START_ACK (0x10)
7. Listen for EVT_STOP_EVENT (0x11) per button
8. Listen for EVT_RACE_COMPLETE (0x12)
9. Write CMD_NEW_RACE:   [0x02, 0x00, 0x00, 0x00]  — to reset
```